

Multi-Leader Replication (Active-Active Writes)

1. What is Multi-Leader Replication?

Multi-leader replication means multiple database nodes can accept write operations simultaneously.

None

Write → Leader A

Write → Leader B

Write → Leader C

Each leader then replicates changes to others.

Also called:

- Multi-primary replication
- Active-active replication
- Master-master replication

2. Why Use It?

Single-leader systems scale reads well, but writes bottleneck at one node.

Multi-leader helps with:

None

Higher write throughput

Multi-region low latency writes

Regional independence

Better availability

3. Architecture Example

None

India Users → Mumbai Leader

EU Users → Frankfurt Leader

US Users → Virginia Leader

Each region writes locally, then syncs globally.

4. Main Benefit = Write Scaling

Instead of one leader processing all writes:

None

`1 node = write bottleneck`

Use:

None

`3 leaders share writes`

5. Biggest Challenge = Consistency Conflicts

If two leaders modify same data simultaneously:

None

`Leader A: price = 100`

`Leader B: price = 120`

Which one wins?

This is the core challenge of multi-leader systems.

6. Why Timestamps Alone Are Dangerous

Different servers have different clocks.

This is called:

None

Clock Skew

Even a few milliseconds difference can disorder writes.

So “latest timestamp wins” can be risky.

7. Common Conflict Scenarios

A. Update vs Update

Two users update same row.

B. Delete vs Update

One node deletes while another edits.

C. Insert vs Insert

Same primary key inserted on two leaders.

D. Counter Increments

Concurrent writes can lose counts if badly handled.

8. Conflict Resolution Strategies

A. Last Write Wins (LWW)

Newest timestamp wins.

Simple but may lose valid updates.

B. Version Numbers

Track revisions and reject stale writes.

C. Merge Logic

Combine fields intelligently.

Example:

None

name from A

address from B

D. Manual Resolution

Send conflict to ops/user.

E. CRDT / Conflict-Free Data Types

Useful for counters, sets, collaborative edits.

9. Read-After-Write Consistency

Important user experience rule:

If user updates data, they should immediately see their own change.

None

User increments counter

Next read should show +1

Even if other users still see old value temporarily.

This is:

None

Read-after-write consistency

10. When Multi-Leader Is Useful

Choose when:

None

Global app with many regions

Need local writes in each region

Temporary disconnected operations

High write throughput

Examples:

- Collaboration apps
- Messaging systems
- Global user profiles
- Edge systems

11. When to Avoid

Avoid when:

None

Strict banking ledger

Strong ordering mandatory

Complex transaction-heavy systems

Use single-leader or consensus systems instead.

12. Real World Examples

- MySQL some master-master setups
- CouchDB multi-master sync
- Microsoft SQL Server merge replication patterns

13. Interview HLD Talking Points

If proposing multi-leader, mention:

- Conflict resolution strategy
- Idempotent writes
- Duplicate detection
- Eventual consistency window
- Regional failover
- Read-your-writes guarantees

14. Comparison

Model	Writes	Complexity
Single Leader	One node	Low
Multi-Leader	Many nodes	High

Leaderless	Many nodes	High
------------	------------	------

15. Short Revision Notes

None

Multi-Leader Replication:

Many leaders accept writes

Pros:

write scaling

regional writes

availability

Cons:

conflicts

clock skew

complexity

16. One-Line Summary

Multi-leader replication scales write traffic by allowing multiple nodes to accept writes, but introduces conflict resolution and consistency challenges.